SKIN CANCER DETECTION SYSTEM

GRADUATION PROJECT DOCUMENTATION (A TO Z)

1. PROJECT OVERVIEW

Skin cancer, specifically melanoma, represents a significant global health risk. Early screening is essential to improve survival rates. However, traditional clinical evaluation using dermoscopic imaging is highly subjective and depends on specialized expert resources.

This graduation project designs and implements an automated, high-precision clinical decision-support system. It utilizes a hybrid deep learning architecture that fuses neural feature representation with gradient-boosted decision trees, and deploys it as a real-time web application dashboard.

2. DATASET & PREPROCESSING

- **Dataset Source:** The benchmark HAM10000 Dataset ('Human Against Machine') containing 10,015 high-quality dermoscopic images of common pigmented skin lesions.
- **Resizing:** All input images are resized to 224 x 224 pixels to match the input requirements of the deep learning architectures.
- **Normalization:** Pixel color values are scaled to the range [0, 1] using Min-Max scaling to stabilize gradient updates.
- **Data Augmentation:** Applied random rotations, zoom, horizontal, and vertical flips during model training to improve generalization on unseen clinical cases.

3. HYBRID MACHINE LEARNING MODELS

Instead of relying on single neural networks, this project introduces a hybrid classifier strategy. Deep neural layers extract high-dimensional visual representation vectors, which are then classified using both neural classifiers and XGBoost.

Pipeline 1: CNN (MobileNetV2) + XGBoost

MobileNetV2 acts as a local feature extractor, converting the skin lesion image into a 1280-dimensional feature vector capturing localized textures, color variegations, and border irregular structures. The feature vector is classified parallelly by a deep neural network dense layer and an XGBoost model. The outputs are fused using the ensemble probability formula:

$$P_{final} = 0.8 * P_{cnn} + 0.2 * P_{xgb} \quad \text{(Malignant Classification Threshold: } t = 0.25\text{)}$$

Pipeline 2: Vision Transformer (ViT) + XGBoost

The Vision Transformer (ViT) divides the image into 49 local patches (7x7 grid) and uses multi-head self-attention mechanisms to map global contextual dependencies across the lesion surface. This feature representation is passed to a ViT head and an XGBoost classifier, fused using the formula:

$$P_{final} = 0.55 * P_{vit} + 0.45 * P_{xgb} \quad \text{(Malignant Classification Threshold: } t = 0.42\text{)}$$

4. ARCHITECTURAL COMPARISON & PERFORMANCE

Feature Comparison	CNN (MobileNetV2) + XGBoost	Vision Transformer (ViT) + XGBoost
Processing Unit	Sliding convolutional filters (kernels)	Image patches + Self-Attention layers

Receptive Field	Local: Small neighborhoods (borders, dots)	Global: Relates distant patches instantly
Inductive Bias	High (assumes pixel neighborhood relevance)	None (learns all spatial structures from scratch)
Key Metric Advantage	Highest Accuracy (84.76%), F1-Score (65.25%)	Highest Diagnostic Recall / Sensitivity (75.40%)
Clinical Use Case	General screening with lower false alerts	Safety-critical screening (catches malignancies)

5. WEB APPLICATION ARCHITECTURE

To make the ML models useful in practice, a high-tech diagnostic HUD (Heads-Up Display) application was developed. The architecture uses a decoupled layout:

- **Frontend Stack:** Built with React and Vite. Tailwind CSS was used to style the sleek, dark, glassmorphic layout. State management is handled globally using React Context (PredictionContext).
- **Backend API:** A fast Python ASGI server using FastAPI running locally on port 8000. It runs model inference on incoming images and sends back classification scores in real-time.
- **Network Tunneling:** A secure LocalTunnel connection (https://skin-cancer-hud-api.loca.lt) bridges the online frontend with the local backend, allowing clinicians to take a photo of a lesion on their phones and run it immediately.

6. CORE APP PAGES & DIAGNOSTIC WORKFLOW

- **Home Hub:** Simulates live calibration telemetry (exposure, sharpness), features a rotating radar scanner, a 3D wrist wireframe, and a selection modal for the diagnostic pipeline.
- **Detect Workspace:** A drag-and-drop area supporting files up to 10MB, a dynamic loader reflecting neural network stages, and API submission.
- **Result Report:** Features a custom visual Split-Slider Viewer comparing the raw lesion side-by-side with a CSS-burn attention heatmap, a semi-circular Risk Gauge, and morphological insights based on the ABCDE clinical rules.
- **Poster Page:** A built-in printable A2-size academic poster presenting the research background, workflow, and model evaluations.

7. SYSTEMS & ENVIRONMENTS (A TO Z)

- **Training:** Google Colab & Anaconda Jupyter environments (PyTorch, PyTorch Image Models, XGBoost).
- **IDE:** Visual Studio Code (VS Code) for system editing.
- **Frontend Local/Hosting:** Node.js (local) and Vercel (production cloud deployment).
- **Backend Local Server:** Anaconda Virtual Environment, served using Uvicorn ASGI on localhost:8000.
- **Tunneling:** LocalTunnel CLI package running in background terminal.